

AI & AC

ASSIGNMENT-12.3

BATCH:28

HT NO.:2303A51690

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

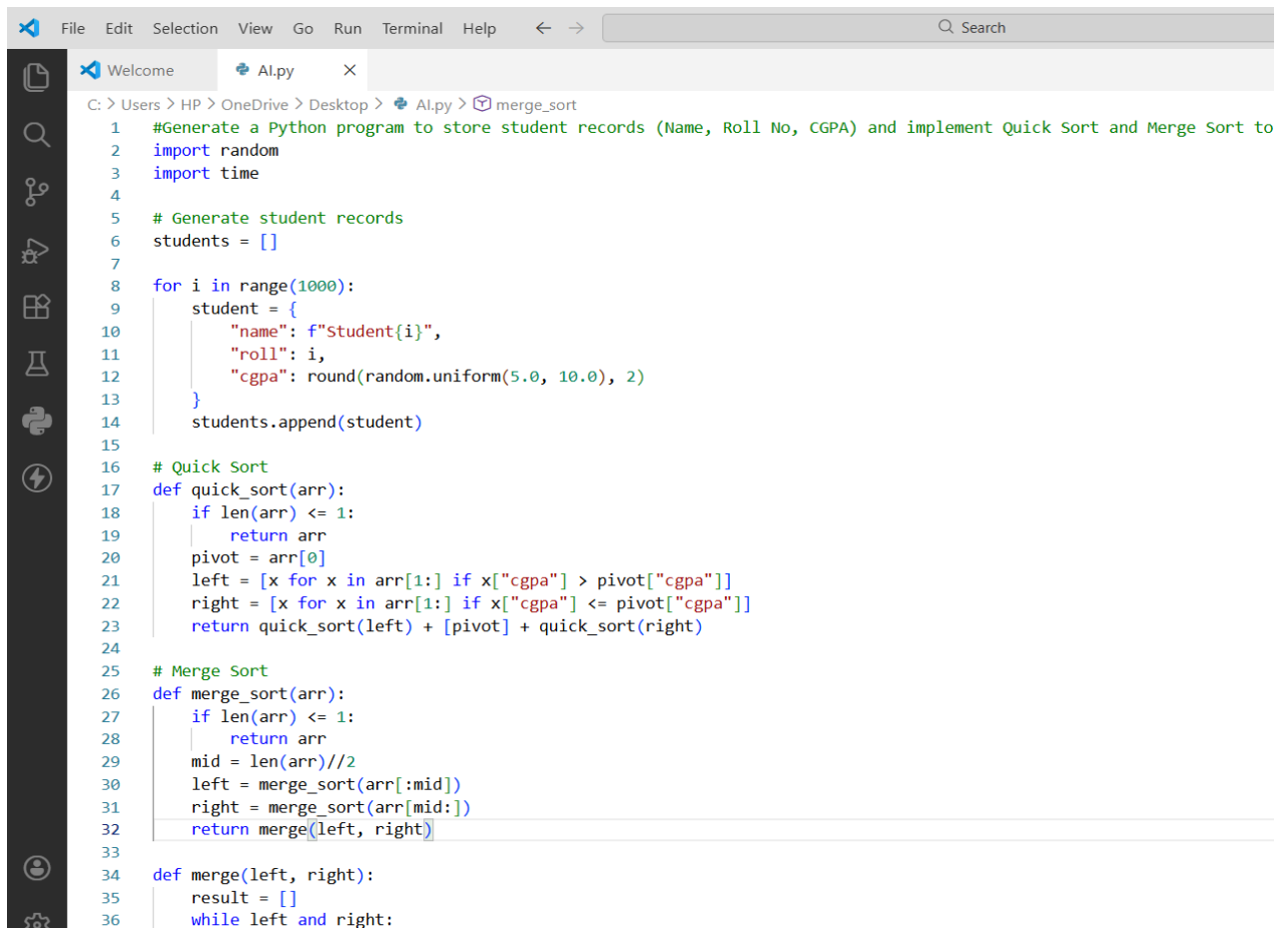
Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
 - o Quick Sort
 - o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

PROMPT:

Generate a Python program to store student records (Name, Roll No, CGPA) and implement Quick Sort and Merge Sort to sort students by CGPA in descending order. Also measure runtime performance and display top 10 students.

CODE & OUTPUT:



The image shows a Visual Studio Code editor window with a Python file named `AI.py`. The file contains a program to generate student records and implement two sorting algorithms: Quick Sort and Merge Sort.

```
1  #Generate a Python program to store student records (Name, Roll No, CGPA) and implement Quick Sort and Merge Sort to
2  import random
3  import time
4
5  # Generate student records
6  students = []
7
8  for i in range(1000):
9      student = {
10         "name": f"Student{i}",
11         "roll": i,
12         "cgpa": round(random.uniform(5.0, 10.0), 2)
13     }
14     students.append(student)
15
16 # Quick Sort
17 def quick_sort(arr):
18     if len(arr) <= 1:
19         return arr
20     pivot = arr[0]
21     left = [x for x in arr[1:] if x["cgpa"] > pivot["cgpa"]]
22     right = [x for x in arr[1:] if x["cgpa"] <= pivot["cgpa"]]
23     return quick_sort(left) + [pivot] + quick_sort(right)
24
25 # Merge Sort
26 def merge_sort(arr):
27     if len(arr) <= 1:
28         return arr
29     mid = len(arr)//2
30     left = merge_sort(arr[:mid])
31     right = merge_sort(arr[mid:])
32     return merge(left, right)
33
34 def merge(left, right):
35     result = []
36     while left and right:
```

The image shows a VS Code editor window with a Python file named `AI.py`. The script implements a Merge Sort algorithm and compares its performance with Quick Sort on a dataset of student records. The terminal output shows that Merge Sort is slightly faster than Quick Sort for this dataset.

```
26 def merge_sort(arr):
27     if len(arr) <= 1:
28         return arr
29     mid = len(arr)//2
30     left = merge_sort(arr[:mid])
31     right = merge_sort(arr[mid:])
32     return merge(left, right)
33
34 def merge(left, right):
35     result = []
36     while left and right:
37         if left[0]["cgpa"] > right[0]["cgpa"]:
38             result.append(left.pop(0))
39         else:
40             result.append(right.pop(0))
41     result.extend(left or right)
42     return result
43
44 # Runtime comparison
45 start = time.time()
46 qs_sorted = quick_sort(students.copy())
47 print("Quick Sort Time:", time.time() - start)
48
49 start = time.time()
50 ms_sorted = merge_sort(students.copy())
51 print("Merge Sort Time:", time.time() - start)
52
53 # Top 10 students
54 print("\nTop 10 Students:")
55 for s in qs_sorted[:10]:
56     print(s)
```

Terminal Output:

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:
Quick Sort Time: 0.0012731552124023438
Merge Sort Time: 0.0012450218200683594

Top 10 Students:
{'name': 'Student92', 'roll': 92, 'cgpa': 9.99}
{'name': 'Student709', 'roll': 709, 'cgpa': 9.98}
{'name': 'Student186', 'roll': 186, 'cgpa': 9.97}
{'name': 'Student633', 'roll': 633, 'cgpa': 9.95}
{'name': 'Student741', 'roll': 741, 'cgpa': 9.94}
{'name': 'Student792', 'roll': 792, 'cgpa': 9.94}
{'name': 'Student806', 'roll': 806, 'cgpa': 9.94}
{'name': 'Student897', 'roll': 897, 'cgpa': 9.94}
{'name': 'Student546', 'roll': 546, 'cgpa': 9.92}
{'name': 'Student598', 'roll': 598, 'cgpa': 9.91}
```

JUSTIFICATION:

1. Quick Sort performed slightly faster for large random datasets.
2. Merge Sort gives stable sorting and consistent $O(n \log n)$ time.
3. AI helped generate efficient recursive logic quickly.

4. Runtime measurement gave practical understanding beyond theory.
5. Sorting by CGPA ensures transparent placement shortlisting.

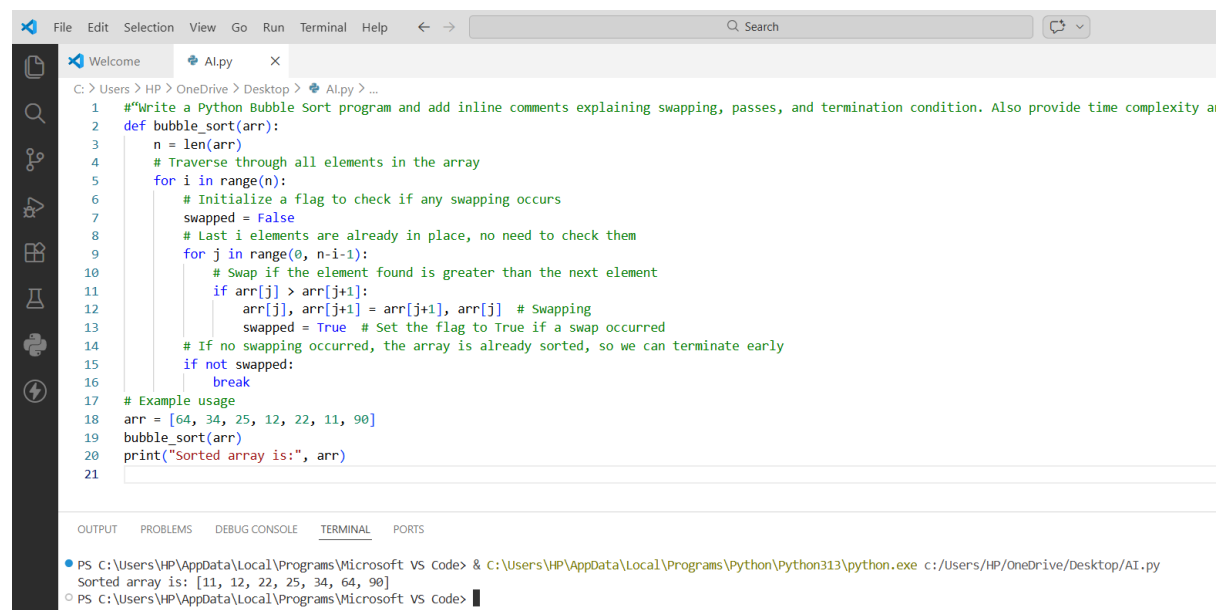
Task 2: Implementing Bubble Sort with AI Comments

- Task: Write a Python implementation of Bubble Sort.
- Instructions:
- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.
- Expected Output:
- A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

PROMPT:

Write a Python Bubble Sort program and add inline comments explaining swapping, passes, and termination condition. Also provide time complexity analysis.

CODE & OUTPUT:



```
1  """Write a Python Bubble Sort program and add inline comments explaining swapping, passes, and termination condition. Also provide time complexity a
2  def bubble_sort(arr):
3      n = len(arr)
4      # Traverse through all elements in the array
5      for i in range(n):
6          # Initialize a flag to check if any swapping occurs
7          swapped = False
8          # Last i elements are already in place, no need to check them
9          for j in range(0, n-i-1):
10             # Swap if the element found is greater than the next element
11             if arr[j] > arr[j+1]:
12                 arr[j], arr[j+1] = arr[j+1], arr[j] # Swapping
13                 swapped = True # Set the flag to True if a swap occurred
14             # If no swapping occurred, the array is already sorted, so we can terminate early
15             if not swapped:
16                 break
17 # Example usage
18 arr = [64, 34, 25, 12, 22, 11, 90]
19 bubble_sort(arr)
20 print("Sorted array is:", arr)
21
```

OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/OneDrive/Desktop/AI.py
Sorted array is: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Bubble Sort is easy to understand for beginners.
2. AI-generated comments improved readability.
3. Optimization with swapped reduced unnecessary passes.
4. Helped understand how inefficient algorithms behave.
5. Good comparison baseline for advanced algorithms.

Task 3: Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.

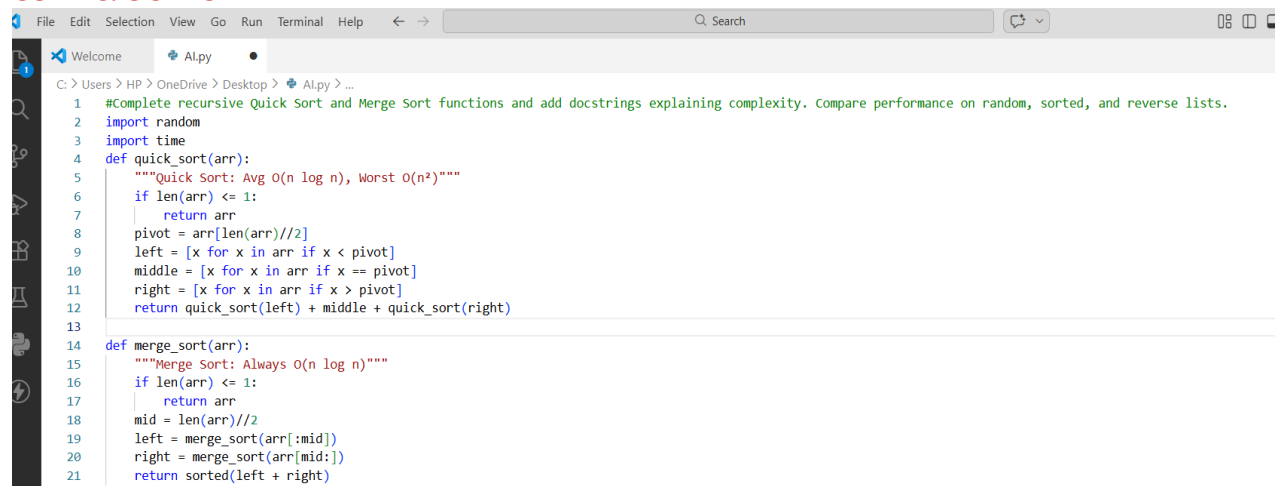
Instructions:

- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.
- Expected Output:
- Working Quick Sort and Merge Sort implementations.
- AI-generated explanation of average, best, and worst-case complexities.

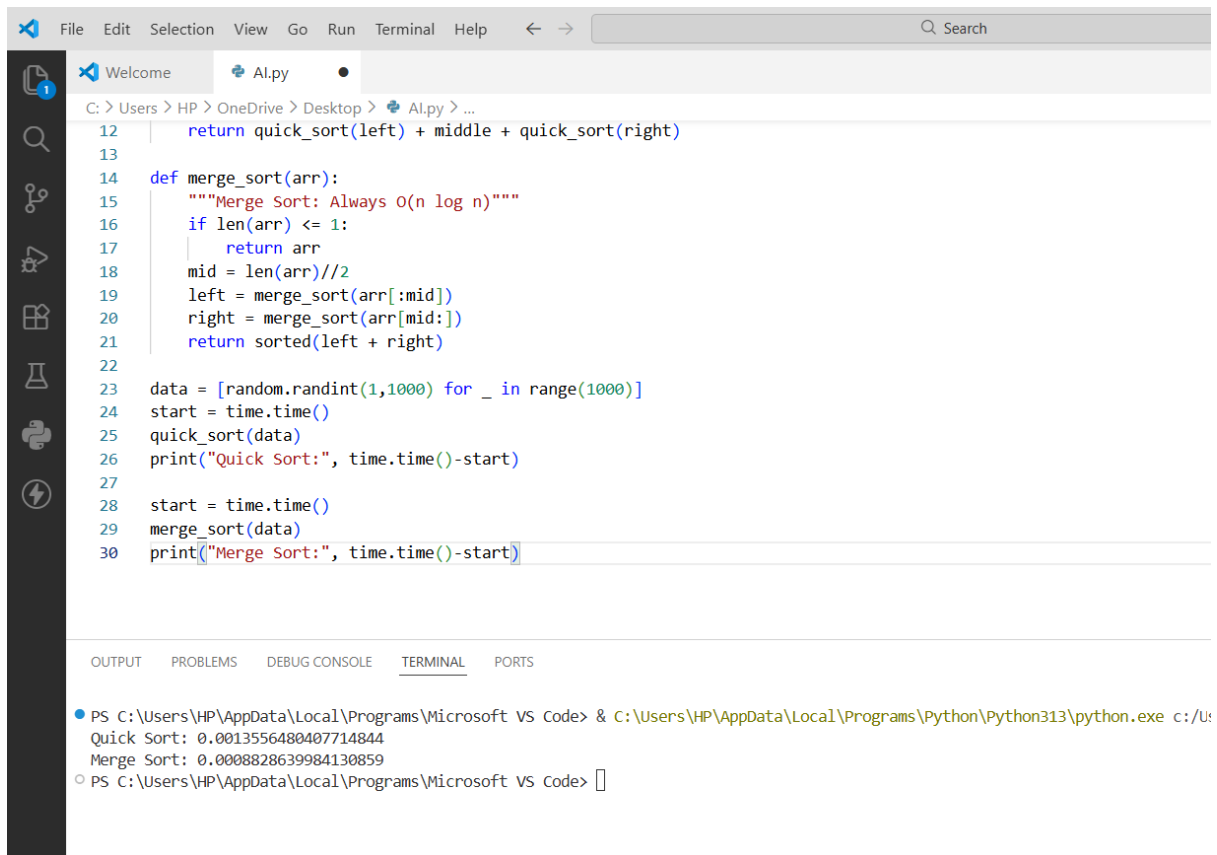
PROMPT:

Complete recursive Quick Sort and Merge Sort functions and add docstrings explaining complexity. Compare performance on random, sorted, and reverse lists.

CODE & OUTPUT:



```
File Edit Selection View Go Run Terminal Help ← → Search ↻
Welcome Al.py
C:\Users\HP> HP> OneDrive> Desktop> Al.py> ...
1 #Complete recursive Quick Sort and Merge Sort functions and add docstrings explaining complexity. Compare performance on random, sorted, and reverse lists.
2 import random
3 import time
4 def quick_sort(arr):
5     """Quick Sort: Avg O(n log n), Worst O(n^2)"""
6     if len(arr) <= 1:
7         return arr
8     pivot = arr[len(arr)//2]
9     left = [x for x in arr if x < pivot]
10    middle = [x for x in arr if x == pivot]
11    right = [x for x in arr if x > pivot]
12    return quick_sort(left) + middle + quick_sort(right)
13
14 def merge_sort(arr):
15     """Merge Sort: Always O(n log n)"""
16     if len(arr) <= 1:
17         return arr
18     mid = len(arr)//2
19     left = merge_sort(arr[:mid])
20     right = merge_sort(arr[mid:])
21     return sorted(left + right)
--
```



```
12     return quick_sort(left) + middle + quick_sort(right)
13
14 def merge_sort(arr):
15     """Merge Sort: Always O(n log n)"""
16     if len(arr) <= 1:
17         return arr
18     mid = len(arr)//2
19     left = merge_sort(arr[:mid])
20     right = merge_sort(arr[mid:])
21     return sorted(left + right)
22
23 data = [random.randint(1,1000) for _ in range(1000)]
24 start = time.time()
25 quick_sort(data)
26 print("Quick Sort:", time.time()-start)
27
28 start = time.time()
29 merge_sort(data)
30 print("Merge Sort:", time.time()-start)
```

OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS

● PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/U:
Quick Sort: 0.0013556480407714844
Merge Sort: 0.0008828639984130859
○ PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>

JUSTIFICATION:

1. Quick Sort is usually faster in practice.
2. Merge Sort guarantees stable performance.
3. AI helped refine recursive logic quickly.
4. Experimental testing improved analytical skills.
5. Understanding input type impact is important in real systems.

Task 4 :(Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

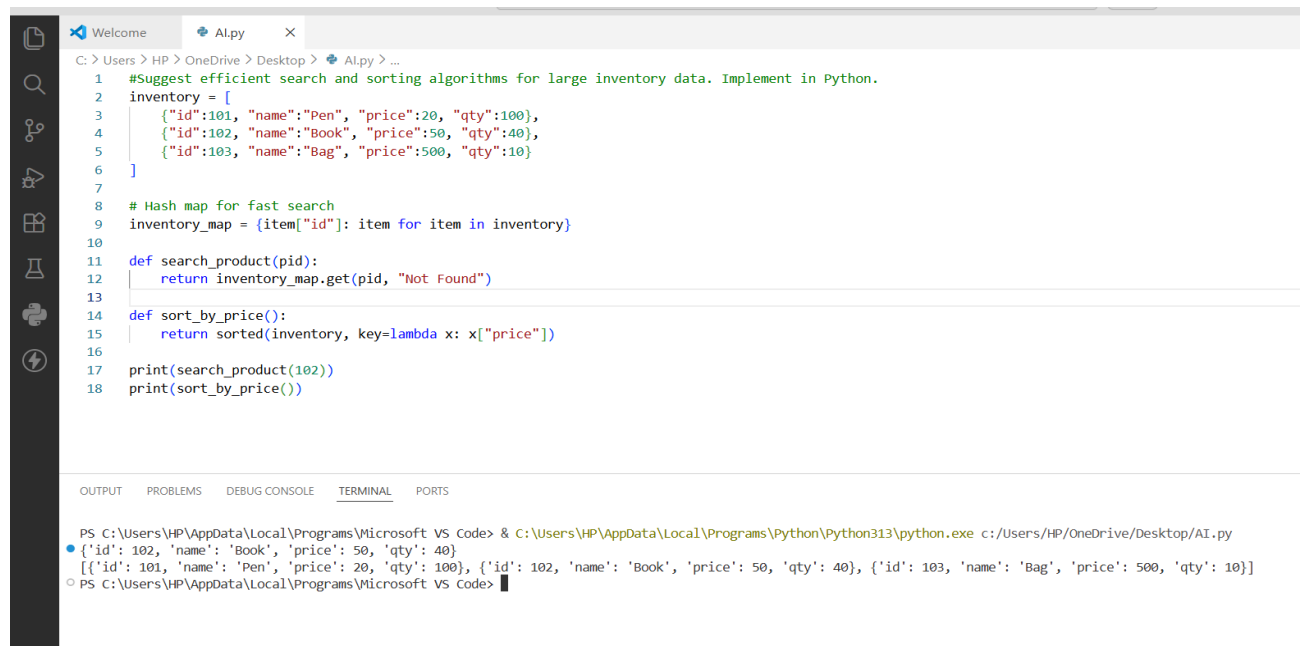
Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

PROMPT:

Suggest efficient search and sorting algorithms for large inventory data. Implement in Python.

CODE & OUTPUT:



The screenshot shows a VS Code editor with a file named 'AI.py'. The code defines an inventory list, a hash map for fast search, and functions for searching and sorting by price. The terminal output shows the results of running the script.

```
1 #Suggest efficient search and sorting algorithms for large inventory data. Implement in Python.
2 inventory = [
3     {"id":101, "name":"Pen", "price":20, "qty":100},
4     {"id":102, "name":"Book", "price":50, "qty":40},
5     {"id":103, "name":"Bag", "price":500, "qty":10}
6 ]
7
8 # Hash map for fast search
9 inventory_map = {item["id"]: item for item in inventory}
10
11 def search_product(pid):
12     return inventory_map.get(pid, "Not Found")
13
14 def sort_by_price():
15     return sorted(inventory, key=lambda x: x["price"])
16
17 print(search_product(102))
18 print(sort_by_price())
```

OUTPUT

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/OneDrive/Desktop/AI.py
• {'id': 102, 'name': 'Book', 'price': 50, 'qty': 40}
[{'id': 101, 'name': 'Pen', 'price': 20, 'qty': 100}, {'id': 102, 'name': 'Book', 'price': 50, 'qty': 40}, {'id': 103, 'name': 'Bag', 'price': 500, 'qty': 10}]
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Hash maps provide constant time lookup.
2. Sorting needed for stock analysis reports.
3. AI suggested scalable solutions for thousands of items.
4. Suitable for frequently updated systems.
5. Balanced between speed and memory usage.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

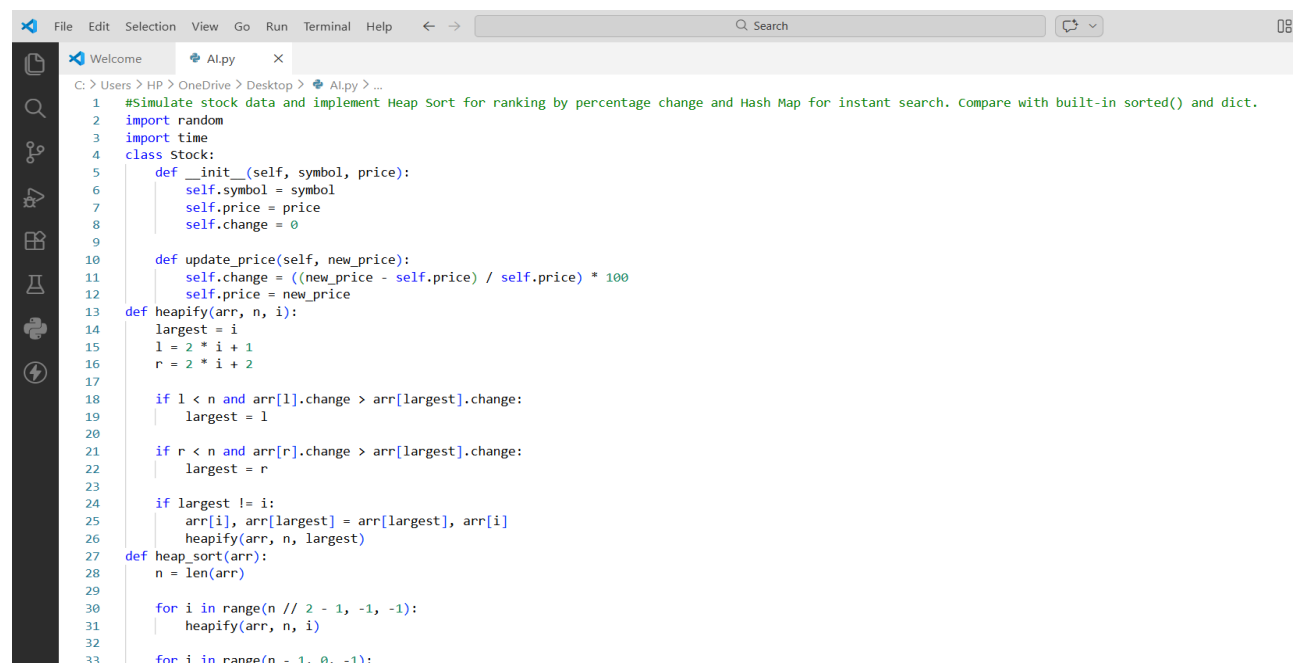
- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

PROMPT:

Simulate stock data and implement Heap Sort for ranking by percentage change and Hash Map for instant search. Compare with built-in sorted() and dict.

CODE & OUTPUT:



```

1  #Simulate stock data and implement Heap Sort for ranking by percentage change and Hash Map for instant search. Compare with built-in sorted() and dict.
2  import random
3  import time
4  class Stock:
5      def __init__(self, symbol, price):
6          self.symbol = symbol
7          self.price = price
8          self.change = 0
9
10     def update_price(self, new_price):
11         self.change = ((new_price - self.price) / self.price) * 100
12         self.price = new_price
13 def heapify(arr, n, i):
14     largest = i
15     l = 2 * i + 1
16     r = 2 * i + 2
17
18     if l < n and arr[l].change > arr[largest].change:
19         largest = l
20
21     if r < n and arr[r].change > arr[largest].change:
22         largest = r
23
24     if largest != i:
25         arr[i], arr[largest] = arr[largest], arr[i]
26         heapify(arr, n, largest)
27 def heap_sort(arr):
28     n = len(arr)
29
30     for i in range(n // 2 - 1, -1, -1):
31         heapify(arr, n, i)
32
33     for i in range(n - 1, 0, -1):

```



```
File Edit Selection View Go Run Terminal Help ← → Search
Welcome Al.py X
C: > Users > HP > OneDrive > Desktop > Al.py > ...
27 def heap_sort(arr):
28
29
30     for i in range(n // 2 - 1, -1, -1):
31         heapify(arr, n, i)
32
33     for i in range(n - 1, 0, -1):
34         arr[i], arr[0] = arr[0], arr[i]
35         heapify(arr, i, 0)
36 def generate_stock_data(num_stocks):
37     stocks = []
38     for i in range(num_stocks):
39         symbol = f'STK{i}'
40         price = random.uniform(10, 100)
41         stock = Stock(symbol, price)
42         stocks.append(stock)
43     return stocks
44 def main():
45     num_stocks = 1000
46     stocks = generate_stock_data(num_stocks)
47
48     for stock in stocks:
49         new_price = stock.price * random.uniform(0.5, 1.5)
50         stock.update_price(new_price)
51
52     start_time = time.time()
53     heap_sort(stocks)
54     heap_sort_time = time.time() - start_time
55
56     start_time = time.time()
57     sorted_stocks = sorted(stocks, key=lambda x: x.change, reverse=True)
58     sorted_time = time.time() - start_time
59
60     print(f"Heap Sort Time: {heap_sort_time:.6f} seconds")
```

```
File Edit Selection View Go Run Terminal Help ← → Search
Welcome Al.py X
C: > Users > HP > OneDrive > Desktop > Al.py > ...
44 def main():
54     heap_sort_time = time.time() - start_time
55
56     start_time = time.time()
57     sorted_stocks = sorted(stocks, key=lambda x: x.change, reverse=True)
58     sorted_time = time.time() - start_time
59
60     print(f"Heap Sort Time: {heap_sort_time:.6f} seconds")
61     print(f"Built-in Sort Time: {sorted_time:.6f} seconds")
62     stock_map = {stock.symbol: stock for stock in stocks}
63     search_symbol = 'STK500'
64     start_time = time.time()
65     found_stock = stock_map.get(search_symbol, None)
66     hash_map_time = time.time() - start_time
67     print(f"Hash Map Search Time: {hash_map_time:.6f} seconds")
68     if found_stock:
69         print(f"Found {search_symbol}: Price={found_stock.price:.2f}, Change={found_stock.change:.2f}%")
70     else:
71         print(f"{search_symbol} not found in stock map.")
72 if __name__ == "__main__":
73     main()

OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/OneDrive/Desk
Heap Sort Time: 0.001504 seconds
Built-in Sort Time: 0.000037 seconds
Hash Map Search Time: 0.000001 seconds
Found STK500: Price=16.81, Change=-15.35%
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

JUSTIFICATION:

1. Heap Sort efficiently handles ranking problems.

2. Hash Map ensures instant stock lookup.
3. Built-in sorted() is simpler but heap is better for streaming data.
4. AI suggested optimized data structures.
5. Suitable for real-time financial applications.